Probability Theory and Machine Learning

Salvador Ruiz Correa

August 10, 2026

This handout provides a unified view of the four foundational topics that drive modern machine learning and deep learning: *approximation theory*, *optimization*, *generalization*, and *representation learning*. Rather than treating these as disjoint subfields, we emphasise their interconnections and, in particular, the *pervasive role of probability theory*. Just as the Bellman equations emerge from conditional independence in a Markov decision process, the tools of probability, concentration inequalities, stochastic processes, information theory, and latent-variable modelling are the glue that transforms each pillar from a set of heuristic recipes into a rigorous science.

The success of modern machine learning rests on four conceptual legs. Approximation theory tells us *what* can be represented; optimisation tells us *how* to find it; generalisation tells us *why* it works on new data; and representation learning tells us *how* to discover the features that make the first three possible. In these notes we explore each pillar in turn, highlighting the probabilistic ideas that underpin them. *Future progress in machine learning, including deep learning, will come from finding ways to integrate and synthesize these perspectives*.

Learning Theory

Machine learning is concerned with learning from data. Consider the usual setup:

1. An unknown probability distribution $P : \mathbb{R}^d \to \{-1, +1\}$.
2. A training sample $x_1, x_2, \cdots, x_N$, with labels $y_1, y_2, \cdots, y_N$; that is, $x_n \in \mathbb{R}$, $y_n \in \{-1, +1\}$, $n = 1, 2, \cdots, N$, and $(x_n, y_n) \sim P$.
3. A loss function $\ell(\hat{y}, y)$, where $\hat{y}$ is our prediction. An example is the square loss $\ell(\hat{y}, y) = \frac{(y\hat{y}-1)^2}{2}$ or the logistic loss, $\ell(\hat{y}, y) = \log(1 + \exp(-y\hat{y}))$.

We care about the population risk, which is

$$R(h) = \mathbb{E}_{(X,Y)\in P}[\ell(h(X), Y)].$$

In practice we calculate the empirical risk from training data:

$$\hat{R}(h) = \frac{1}{N} \sum_{n=1}^{n} \ell(h(x_n), y_n).$$

Our goal is to provide an algorithm that learns from a reasonable (polynomial) amount of data and achieves low risk (low empirical risk).

We need to assume how the labels are chosen (realizable case), that is, we assume that there is a set (or class) of functions $\mathcal{H}$ (hypothesis space) such that $y = h^*(x)$ for some $h^*(x) \in \mathcal{H}$.

A more general case consideras a hypotheisis space $\mathcal{H}$ of functions $h : \mathbb{R}^d \to \mathbb{R}$ such that

$$\mathbb{E}[Y \mid X] = h(X).$$

Thus, the label is no longer a deterministic function of the feature vector, but crucially, the noise has expectation zero.

Approximation Theory (Expressivity)

The question: Can a chosen model class (e.g., a neural network architecture) represent the target function we wish to learn? This is the classical problem of *expressivity* or *capacity*.

Classical results. The Universal Approximation Theorem states that a feedforward network with a single hidden layer and a non-polynomial activation can approximate any continuous function on a compact set to arbitrary accuracy, provided the layer is wide enough. More recent work investigates the trade-off between depth and width, the efficiency of approximation (how many parameters are needed for a given error ϵ), and the curse of dimensionality.

Curse of Dimensionality: In high dimensions, the volume of the space grows so fast that covering it with a grid of points requires an exponential number of samples. Classical approximation fails here.

The role of probability. Classical approximation theory often uses worst-case (uniform) error metrics, e.g., the supremum norm $\|f - \hat{f}\|_\infty$. However, machine learning cares about *average-case* performance with respect to the data-generating distribution $P(X, Y)$. Probability reframes the problem as minimising an expected loss:

$$\mathbb{E}_{(X,Y)\sim P}[\ell(h(X), Y)] = \int \ell(h(x), y)\, dP(x, y).$$

Regions of the input space with low probability density contribute negligibly to the error. This perspective enables *high-dimensional* approximation theorems (e.g., Barron's theorem) that circumvent the curse of dimensionality[1] by exploiting the smoothness and concentration of the data distribution, rather than requiring exact pointwise fidelity over the entire high-dimensional space.

[1] Barron's theorem shows that neural networks can approximate functions with bounded Fourier spectrum using a number of parameters that scales with the complexity of the function, not the input dimension—provided we measure error under the data distribution, not uniformly.

Optimization

The question: Given a model that can represent the target, how do we find the specific parameters θ that minimise a chosen loss func-

tion? In deep learning this is a highly non-convex, high-dimensional problem:

$$\theta^* = \arg\min_{\theta} \left\{ \frac{1}{N} \sum_{i=1}^{N} \ell(h_\theta(x_i), y_i) + \lambda\Omega(\theta) \right\}.$$

The loss landscape is rugged, with saddle points and local minima. Optimisation studies convergence rates, stability, and implicit biases of iterative algorithms (primarily gradient descent and its variants).

The role of probability. Probability is the engine that makes optimization tractable at scale. The workhorse is **Stochastic Gradient Descent (SGD)**, where gradients are estimated from random mini-batches:

$$\theta_{t+1} = \theta_t - \eta_t \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} \nabla \ell(h_{\theta_t}(x_i), y_i).$$

This injects stochastic noise into the update. Convergence analysis relies heavily on probabilistic tools:

- **Martingale theory** for almost-sure convergence under decreasing step-sizes.
- **Concentration inequalities** (e.g., Bernstein) to bound the deviation of the stochastic gradient from the true batch gradient.[2]
- **Stochastic Differential Equations (SDEs)** model the continuous-time limit of SGD;[3] the injected Gaussian noise helps the algorithm escape sharp minima in favour of flatter, more generalisable ones—a phenomenon known as the *implicit regularisation* of SGD.

Martingale: A sequence of random variables where the expected next value, given the past, is exactly the current value (like a "fair game"). It helps prove that SGD doesn't systematically drift away.

[2] Concentration inequalities (like Hoeffding's or Bernstein's) give us probabilistic bounds on how much the average of a random sample can deviate from the true expectation. They are the workhorses of statistical guarantees.

[3] SDEs are like ordinary differential equations but with an added random noise term. They allow us to analyze SGD as a continuous physical process, revealing that the noise helps the algorithm find "flat" (generalizable) minima rather than "sharp" ones.

Generalization

The question: Why does a model that fits the training data (often perfectly) still perform well on unseen data? This is the core of statistical learning theory. Modern neural networks are massively overparameterised; they have millions of parameters and can fit random labels.yet they generalise. Generalization theory investigates complexity measures (VC-dimension, Rademacher complexity), algorithmic stability, and the simplicity bias of gradient descent.

The role of probability. Generalization is irreducibly probabilistic. It rests on the assumption that training and test data are drawn i.i.d. from the same unknown distribution $P(X, Y)$. The goal is to provide **Probably Approximately Correct (PAC)** guarantees. The true risk is

$$R(h) = \mathbb{E}_{(X,Y)\sim P}[\ell(h(X), Y)],$$

PAC Learning: "Probably Approximately Correct" means that with high probability (the *Probably*), the model we output will have a small test error (the *Approximately Correct*).

while the empirical risk is $\hat{R}(h) = \frac{1}{N}\sum_i \ell(h(x_i), y_i)$. Probability bridges these via **concentration inequalities**:

$$P\left(\sup_{h\in\mathcal{H}} |R(h) - \hat{R}(h)| > \epsilon\right) \leq \delta,$$

where the bound depends on the complexity of $\mathcal{H}$. Hoeffding's inequality gives finite-sample bounds for finite classes; Rademacher complexity extends this to infinite classes.[4] The classic **bias–variance trade-off** is a direct probabilistic decomposition of the expected test error, explicitly separating approximation error (bias) from estimation error (variance).

[4] **Rademacher Complexity:** A measure of how well a hypothesis class can fit pure random noise. If a model class has high Rademacher complexity, it can memorize random labels, meaning it is too flexible and may not generalize.

Representation Learning

The question: How can a model automatically discover the features (representations) needed for a task, rather than relying on handcrafted ones? Deep learning excels at this: hierarchical layers transform raw pixels or text into increasingly abstract, semantically meaningful latent spaces. Representation learning studies unsupervised pretraining (autoencoders), self-supervised learning (contrastive methods), and generative modelling.

The role of probability. Probability provides the normative framework for what constitutes a "good" representation. This is formalised through **latent variable models**. The observed data X is assumed to be generated from an unobserved latent Z via a conditional likelihood $p_\theta(x \mid z)$ and a prior $p(z)$. Representation learning is the inverse problem: inferring the posterior $p(z \mid x)$.

This viewpoint underpins several cornerstones:

- **Variational Autoencoders (VAEs)** maximise the Evidence Lower Bound (ELBO).[5] This uses the **Kullback–Leibler (KL) divergence** to regularise the learned latent distribution $q_\phi(z \mid x)$ towards the prior $p(z)$.
- The **Information Bottleneck (IB)** principle frames representation learning as a trade-off:

  $$\mathcal{L} = I(Z;X) - \beta I(Z;Y),$$

  where $I(\cdot;\cdot)$ is Shannon **mutual information**.[6] This explains why deep networks discard irrelevant input variations while preserving predictive signals.
- **Contrastive learning** (e.g., SimCLR, InfoNCE) maximizes a lower bound on the mutual information between different augmented views of the same sample. [7]

[5] **ELBO:** The "Evidence Lower BOund" is a tractable objective that we maximize instead of the intractable true log-likelihood. It encourages the encoder to match the prior and the decoder to reconstruct the data.
KL Divergence: A measure of how one probability distribution differs from another. If P and Q are distributions, $D_{KL}(P||Q)$ tells us the information loss when we use Q to approximate P.

[6] **Mutual Information** $I(X;Y)$**:** Measures the reduction in uncertainty about X given Y (or vice versa). $I(X;Y) = 0$ means they are independent. It quantifies how much information Y carries about X.

Summary: The Four Pillars at a Glance

[7] **InfoNCE:** This loss function turns representation learning into a classification problem. It asks the model to pick the correct "positive" pair among a set of "negative" distractors. Maximizing this accuracy is equivalent to estimating mutual information.

Pillar	Core Question	Probabilistic Tool(s)
Approximation	Can the model represent the target function?	Average-case error (integrals w.r.t. data distribution); circumvents the curse of dimensionality.
Optimization	How to find the optimal parameters efficiently?	SGD, martingales (fair games), concentration inequalities (tail bounds), SDEs (continuous noisy dynamics).
Generalization	Why does training success transfer to new data?	PAC learning (high-probability guarantees), Rademacher complexity (noise-fitting capacity), bias–variance.
Representation	How to discover useful features automatically?	Latent variables, KL divergence (distribution distance), mutual information (dependency), ELBO (tractable objectives), InfoNCE.

In summary, probability theory is not merely an auxiliary tool but the common language that unifies the four pillars. Approximation is measured in expectation; optimisation is driven by stochastic gradients; generalisation is bounded by probabilistic tail inequalities; and representation learning is grounded in information-theoretic objectives. Just as the Bellman equations leverage the conditional independence structure of an MDP to compute value functions without explicit marginalisation, these probabilistic frameworks allow us to tackle high-dimensional, data-driven problems with rigorous guarantees and efficient algorithms.

Further Reading

- Barron, A. R. (1993). Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Trans. Inf. Theory*.
- Bottou, L., Curtis, F. E., & Nocedal, J. (2018). Optimization methods for large-scale machine learning. *SIAM Review*.
- Vapnik, V. N. (1998). *Statistical Learning Theory*. Wiley.
- Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE Trans. PAMI*.

Probability Theory Foundations

KOLMOGOROV'S AXIOMS are the foundations of probability theory. These axioms were introduced by Andrey Kolmogorov in 1933. These axioms remain central and have direct contributions to real-world probability cases.

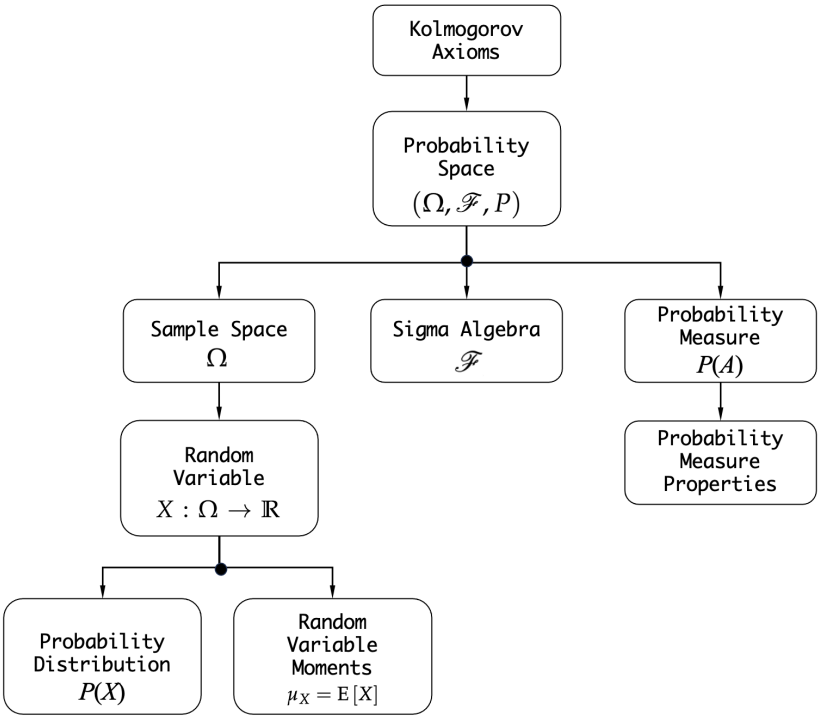

Figure 1: Key Kolmogorov probability concepts.